

Report Tecnologie e Servizi WEB

Componenti del gruppo:

- Bedino Edoardo - matricola: 997518
- Candela Davide - matricola: 976529
- Kostadinov Iliyan - matricola: 979118

Link alla repo **GitLab**: <https://gitlab2.educ.di.unito.it/st214128/assignment-ium-tweb>

NOTA: Noi siamo il gruppo che le ha mandato la mail relativa al problema di rinominare la main directory con i nostri cognomi. Non siamo riusciti a trovare una soluzione e per tanto quando scaricherà da GitLab il nostro progetto, esso sarà contenuto in una directory chiamata nel seguente modo: *assignment-ium-tweb*.

Introduzione:

Il seguente report è suddiviso per: sviluppo frontend, routes implementate e socket.

Poiché il nostro modo di spartire, per ogni membro, il lavoro è stato:

- Scegliere una pagina della web-app (esempio pagina dei Players)
- Sviluppare la specifica pagina graficamente
- Scrivere le api per comunicare con il main server
- Scrivere le route necessarie al raggiungimento del proprio obiettivo
- Scrivere le query d'interesse sul server Express e Springboot per prelevare le informazioni necessarie

In questo modo la suddivisione del lavoro riguarda un campo d'interesse che non si interseca con gli interessi altrui.

Ognuno di noi è indipendente e sviluppa le proprie *API* in modo autonomo.

Alla termine del lavoro i vari server saranno arricchiti con diverse *Routes* che calcolano e scambiano dati o con gli altri due server di supporto (coloro che possono interfacciarsi con i *Database*) oppure rispondono alle specifiche richieste del *Client*.

Technical Tasks:

- Sviluppo del frontend
 - **Solution:** La soluzione adottata per sviluppare l'interfaccia grafica della web-app è stata utilizzare una libreria javascript di frontend chiamata React. Con React possiamo creare delle componenti grafiche che interagiscano tra di loro e con l'utente. Il vantaggio di utilizzare questa libreria è che il codice diventa molto più pulito e ordinato, ogni componente ha delle sue responsabilità ed azioni da eseguire. Abbiamo anche utilizzato una libreria grafica basata su React chiamata NextUI, la quale ci fornisce alcune componenti già predefinite con uno stile unico e innovativo. Inoltre abbiamo utilizzato TailwindCSS per un'esperienza migliore con il CSS.
 - **Issues:** Ci siamo ritrovati ad affrontare un argomento nuovo non trattato a lezione, quindi abbiamo utilizzato dei corsi online e risorse in rete per imparare ad usare React. All'inizio è stato un po' difficile entrare nell'ottica della programmazione a componenti ma poi con il passare del tempo ci siamo adattati in fretta.
 - **Requirements:** Nei requisiti dell'assignment è richiesto di creare pagine HTML+CSS+Javascript per il front-end e noi abbiamo rispettato questa specifica poiché React è costruita su Javascript, mentre TailwindCSS è basato su CSS.

- **Limitations:** Siccome è stata la nostra prima volta nell'utilizzare una libreria a componenti grafiche, siamo riusciti solo parzialmente a costruire componenti di uso generale in modo da essere riutilizzabili anche in altre pagine future (in ottica futura di estensione della web-app). Durante la navigazione all'interno delle pagine della web-app, quando si vuole tornare indietro non viene salvato lo stato precedente della pagina.
- Routes e query per la pagina dei Players
 - **Solution:** La soluzione consiste nel chiedere al *main server* su due *routes* differenti una lista di players: 1) la prima lista riguarda i 9 players più costosi nell'ultima stagione calcistica (i top players). 2) la seconda lista è un elenco di players filtrati. I players possono essere filtrati per nome, nome squadra e ruolo. Il *main server* provvederà a chiedere agli altri due server di eseguire calcoli ed ottenere informazioni dal *Database* per completare le cards relative a ciascun player richiesto/filtrato.
 - **Issues:** Il problema maggiore riscontrato è stato quello di implementare la funzionalità del filtraggio player in maniera corretta.
 - **Requirements:** Le informazioni sui players, sono state salvate sul *Database Postgres*. Il *Springboot server* ha il compito di occuparsi di ottenere tali informazioni utilizzando la libreria *JPA*.
 - **Limitations:** I server che gestiscono le richieste per ottenere le informazioni sui players non effettuano controlli sui dati. Questa responsabilità è delegata alla web-app, i dati vengono adattati alla specifica visualizzazione.
- Routes e query per la pagina dei Clubs
 - **Solution:** La soluzione consiste nel chiedere al *main server* su due *routes* differenti una lista di clubs: 1) la prima lista riguarda i 9 clubs più popolari nella storia calcistica. 2) la seconda lista è un elenco di clubs filtrati. I clubs possono essere filtrati per nome, campionato e nazione. Il *main server* provvederà a chiedere agli altri due server di eseguire calcoli ed ottenere informazioni dal *Database* per completare le cards relative a ciascun club richiesto/filtrato.
 - **Issues:** Il problema maggiore riscontrato è stato quello di implementare la funzionalità del filtraggio club in maniera corretta. In particolare se è presente il filtro sulla competizione non dovrebbe essere presente quello sulla nazione, mentre il viceversa è concesso.
 - **Requirements:** Le informazioni sui clubs sono state salvate sul *Database Postgres*. Il *Springboot server* ha il compito di occuparsi ed ottenere tali informazioni utilizzando la libreria *JPA*.
 - **Limitations:** I server che gestiscono le richieste per ottenere le informazioni sui clubs non effettuano controlli sui dati. Questa responsabilità è delegata alla web-app, infatti i dati vengono adattati alla specifica visualizzazione. Le voci nella tendina per il filtraggio sulla nazione sono in locale e non collegate al *Database*.
- Routes e query per la pagina delle Competitions
 - **Solution:** La soluzione consiste nel chiedere al *main server* su due *routes* differenti una lista di competitions: 1) la prima lista riguarda le 9 competition più popolari e seguite 2) la seconda lista è un elenco di competitions. Le competitions possono essere filtrate per nome, nazione e tipologia. Il *main*

server provvederà a chiedere agli altri due server di eseguire calcoli ed ottenere informazioni dal *Database* per completare le cards relative a ciascuna competition richiesta/filtrata.

- **Issues:** Scelta implementativa sul filtraggio della tipologia (type/sub-type).
- **Requirements:** Le informazioni sulle competitions, sono state salvate sul *Database Postgres*. Il *Springboot server* ha il compito di occuparsi ed ottenere tali informazioni utilizzando la libreria *JPA*.
- **Limitations:** I server che gestiscono le richieste per ottenere le informazioni sui clubs non effettuano controlli sui dati. Questa responsabilità è delegata alla web-app, i dati vengono adattati alla specifica visualizzazione. Le voci nella tendina per il filtraggio sulla tipologia e la nazione sono in locale e non collegate al *Database*.

- Routes e query per il modal del Ranking

- **Solution:** la soluzione consiste nel chiedere al *main server* su una *route* di calcolare la classifica di una competizione con tipologia domestic-league e restituire le squadre ordinate per punteggio totale, con relative informazioni sui goals totali fatti e subiti. Il *main server* provvederà a chiedere all'*Express server* di ottenere le informazioni sulle partite necessarie per calcolare la classifica.
- **Issues:** -
- **Requirements:** Le informazioni sulle partite sono state salvate sul *Database Mongo*. L'*Express server* ha il compito di occuparsi di ottenere tali informazioni utilizzando *moongose*.
- **Limitations:** -

- Routes e query per la pagina dei Games

- **Solution:** La soluzione consiste nel chiedere al *main server* su una *route* una lista di games. Il *main server* provvederà a fare una prima richiesta all'*Express server* per ottenere una lista filtrata per competizione, anno e round. Successivamente per ogni partita il *main server* richiede al *Springboot server* le informazioni relative alle squadre coinvolte.
- **Issues:** Combinare le due risposte dei server secondari in modo da ottenere un'unica risposta contenente tutte le informazioni necessarie.
- **Requirements:** Le informazioni sui games, sono state salvate sul *Database Mongo*. L'*Express server* ha il compito di occuparsi di ottenere tali informazioni utilizzando *moongose*.
- **Limitations:** I server che gestiscono le richieste per ottenere le informazioni sui games non effettuano controlli sui dati. Questa responsabilità è delegata alla web-app, infatti i dati vengono adattati alla specifica visualizzazione.

- Routes e query per la pagina dei GameEvents e GameLineUps

- **Solution:** La soluzione consiste nel chiedere al *main server* su due *routes* differenti una lista di game events riguardanti una specifica partita giocata e anche la game-lineup della stessa. Il *main server* provvederà a chiedere all'*Express server* i dati riguardanti gli eventi e la formazione della partita. Successivamente per ogni giocatore coinvolto nella partita il *main server* richiede al *Springboot server* le informazioni relative ai players.
- **Issues:** Disporre graficamente gli eventi e la formazione della partita.

- **Requirements:** Le informazioni sui game events e lineups, sono state salvate sul *Database Mongo*. L'*Express server* ha il compito di occuparsi di ottenere tali informazioni utilizzando *moongose*.
 - **Limitations:** I server che gestiscono le richieste per ottenere le informazioni sui game events e game-lineups non effettuano controlli sui dati. Questa responsabilità è delegata alla web-app, infatti i dati vengono adattati alla specifica visualizzazione.
- Socke.IO e chat
- **Solution:** La soluzione consiste nell'utilizzare la libreria Socket-IO sul *main server* la quale ci permette di aprire comunicazioni utilizzando i socket. In questo modo si può ottenere uno scambio di messaggi in tempo reale per realizzare una chat tra utenti. Nella web-app la pagina della chat è suddivisa in diverse rooms, dando la possibilità all'utente di scegliere in quale entrare.
 - **Issues:** Realizzare graficamente una pagina che permetta di visualizzare le chat e i messaggi. Connettere la web-app con il *main server* poiché il nostro metodo è stato differente da quello presentato durante le lezioni di laboratorio.
 - **Requirements:** Utilizzo della libreria Socket-IO e implementazione di rooms come nelle lezioni seguite in laboratorio.
 - **Limitations:** Poiché l'utente non effettua una vera registrazione/login al *main server*, i messaggi non vengono salvati in modo persistente.

Conclusioni e divisione del lavoro:

Bedino: pagina dei games, game-events e lineups, chat

Candela: pagina dei clubs, pagina delle competitions

Kostadinov: home-page, pagina dei players, pagina del ranking (l'API è stata scritta da Candela)

Riteniamo di aver suddiviso il lavoro in maniera equa e flessibile. Alla fine ogni membro del gruppo ha dovuto sviluppare 2/3 pagine grafiche con relative *API*, scrivere le *routes* sui server d'interesse e costruire la *query* (mongo o sql) in base alle informazioni necessarie per popolare la sua pagina sviluppata. Chiaramente c'era la possibilità in caso di necessità che qualche membro dovesse prestare aiuto in qualche bisogno specifico di un compagno.

In ogni caso il grosso del lavoro è stato quello di costruire graficamente le diverse pagine in modo da ottenere una visualizzazione dinamica e innovativa. Seguendo le lezioni dei laboratori abbiamo implementato i diversi server in maniera corretta. Abbiamo utilizzato *Axios*, per fare le chiamate asincrone tra il client e i vari server, e *Async/Await* per gestirle.

Bibliografia:

<https://react.dev/>

<https://nextui.org/>

<https://tailwindcss.com/>

<https://chatgpt.com/>